Karla Sahin

Carnegie Mellon University

B.S. Electrical & Computer Engineering + Robotics '28

I build and test robotic systems across the stack — from ROS 2 calibration and perception on warehouse robots to multi-robot planning, bare-metal firmware, and FPGA logic.

Engineering portfolio · Summer 2027 internships in robotics & embedded software

karlasahin0@gmail.com
klsahin.github.io · github.com/klsahin
linkedin.com/in/karla-sahin



fig. 01 Locus Array

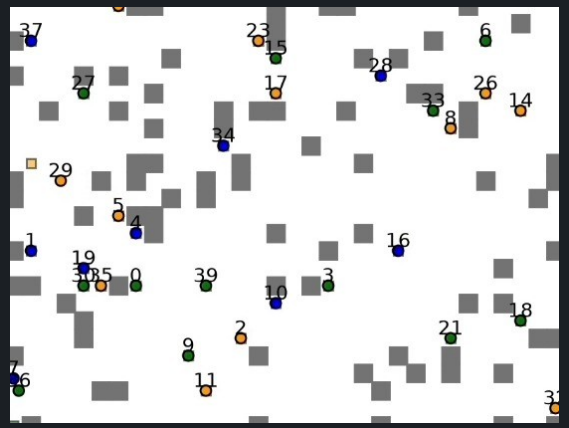


fig. 02 multi-robot PBS

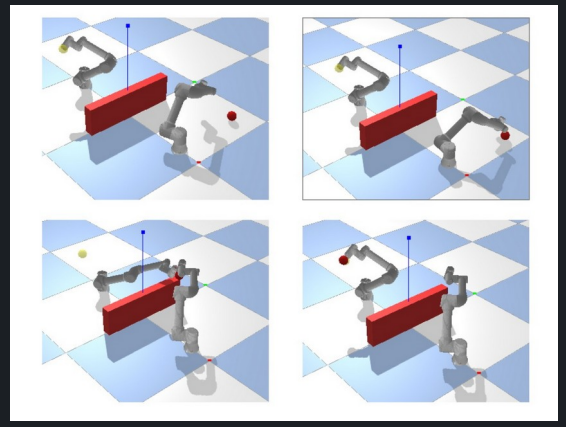


fig. 03 UR5e handoff

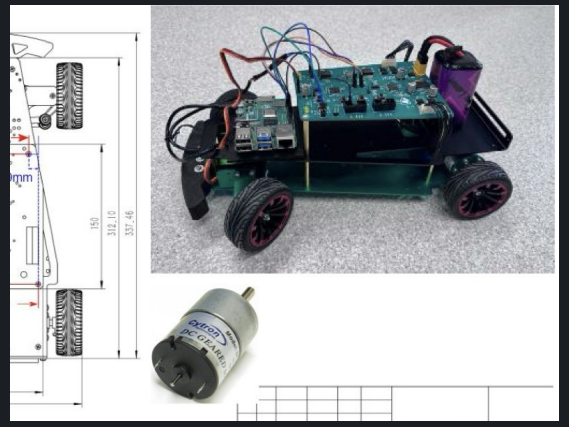


fig. 04 STM32 robot car

Six projects, one throughline: software on real hardware

Robots in the field

Built a hand-eye calibration system and Gazebo simulations of the picking system for a production warehouse robot.

Planning under uncertainty

From 7-DoF arm planners to multi-robot fleets that lose communication.

Down to the metal

Bootloaders, MMIO drivers, interrupts, and FPGA RTL.

Project	Perception & sensing	Motion & multi-robot planning	Embedded firmware	Digital hardware	Test & dev tooling	Main stack	p.
Calibration validation · Locus Robotics	●		●		●	ROS 2 · OpenCV	3
Multi-robot assembly planner (SPAR)		●			●	RRT* · RL · MAPF	4
Dual-arm motion planning		●			●	C++ · PyBullet	5
Bare-metal ARM kernel	●		●			C · ARM asm	6
FPGA RTL game system				●		SystemVerilog	7
Xspine sensing research (CHI '26)	●		●			Python · Unity	8

Each project page follows: outcome → my contribution → skills → technical detail.

Catching camera–arm calibration drift without recalibrating

Locus Array picks items with a 6-DoF arm guided by a multi-camera rig. **I built the ROS 2 validation that measures whether the cameras and the arm's kinematics still agree**, plus OpenCV tools to diagnose drift with no recalibration or added hardware.

WHAT I BUILT

- Hand-eye check: fiducial detection + PnP vs. arm pose via TF composition
- Reprojection-error and point-cloud overlay visualizers in OpenCV
- Ran on Hardware-in-the-Loop benches with NVIDIA Orins

`ROS 2` `OpenCV` `PnP` `TF` `HIL`



fig. 01 Locus Array

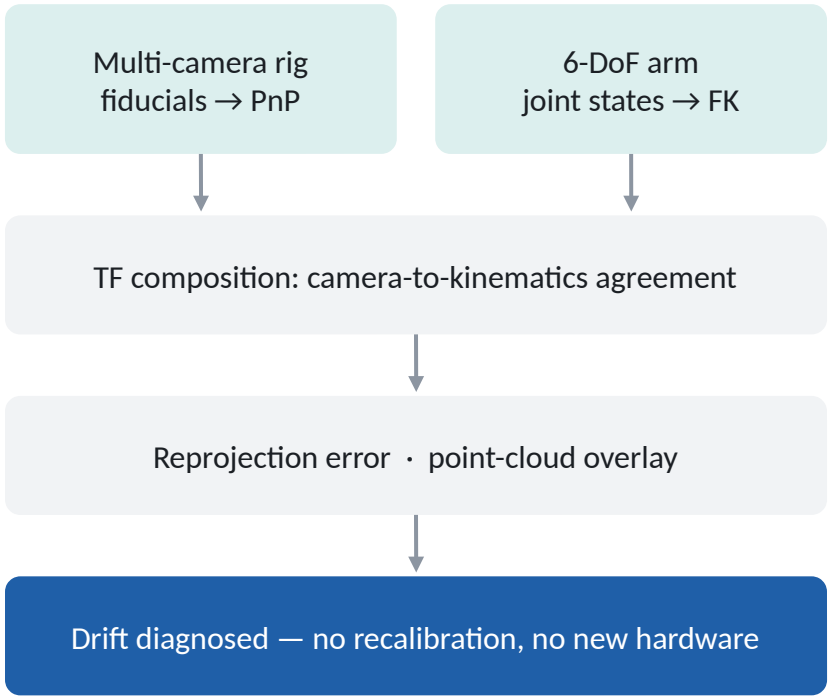


fig. 02 validation pipeline I built

ALSO SHIPPED AT LOCUS

Fleet dashboards
Grafana / VictoriaMetrics: network health + pick performance at customer sites

Controller telemetry
Controller-board data integrated into ROS for lighting control

Docs in CI
State-machine diagrams auto-generated so docs track code

Gazebo sim
Arm + camera rig in sim for hardware-free development

Keeping a free-flying robot fleet on task when links drop

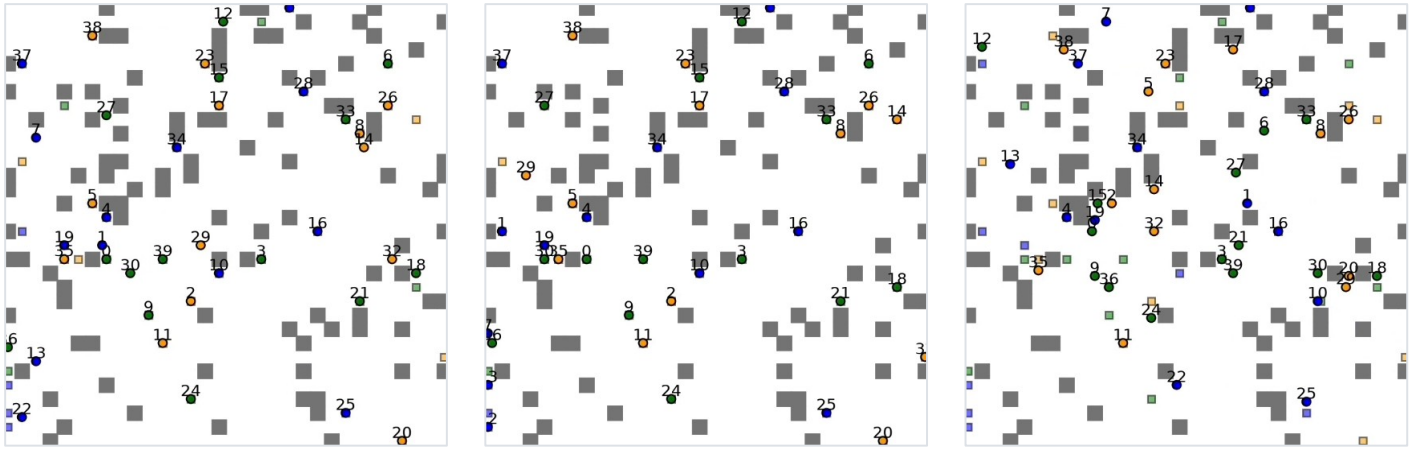


fig. 03 Priority-Based Search coordinating a multi-agent fleet around obstacles

PLANNER ARCHITECTURE

Distributed priority negotiation → Kinodynamic RRT* paths → Reactive fallback control

Execution-time repair: RL-guided task reassignment · retiming · rerouting

triggered by comm loss, sensor drift, partially known maps

In-space assembly needs many robots working together, but localization is unreliable and links drop. **I'm designing a decentralized planner that keeps the fleet coordinated anyway** — and repairs the plan mid-execution when things go wrong.

MY FOCUS

- Negotiate priorities between robots without a central planner
- Plan dynamically feasible paths for free-flyers with kinodynamic RRT*
- Fall back to reactive control when the plan can't be trusted

`MAPF` `PBS` `Kinodynamic RRT*` `RL` `Multi-robot`

Two UR5e arms, one object: planning a collision-free handoff

Implemented four sampling-based planners in C++ for arms up to 7-DoF, then used them to coordinate two UR5e arms that pick, hand off, and place an object around a wall in PyBullet.

PLANNER SUITE

RRT
single-tree baseline

RRT-Connect
bidirectional, fast to first path

RRT*
rewires toward shorter paths

PRM
reusable roadmap, multi-query

`C++` `PyBullet` `Collision checking` `Up to 7-DoF`

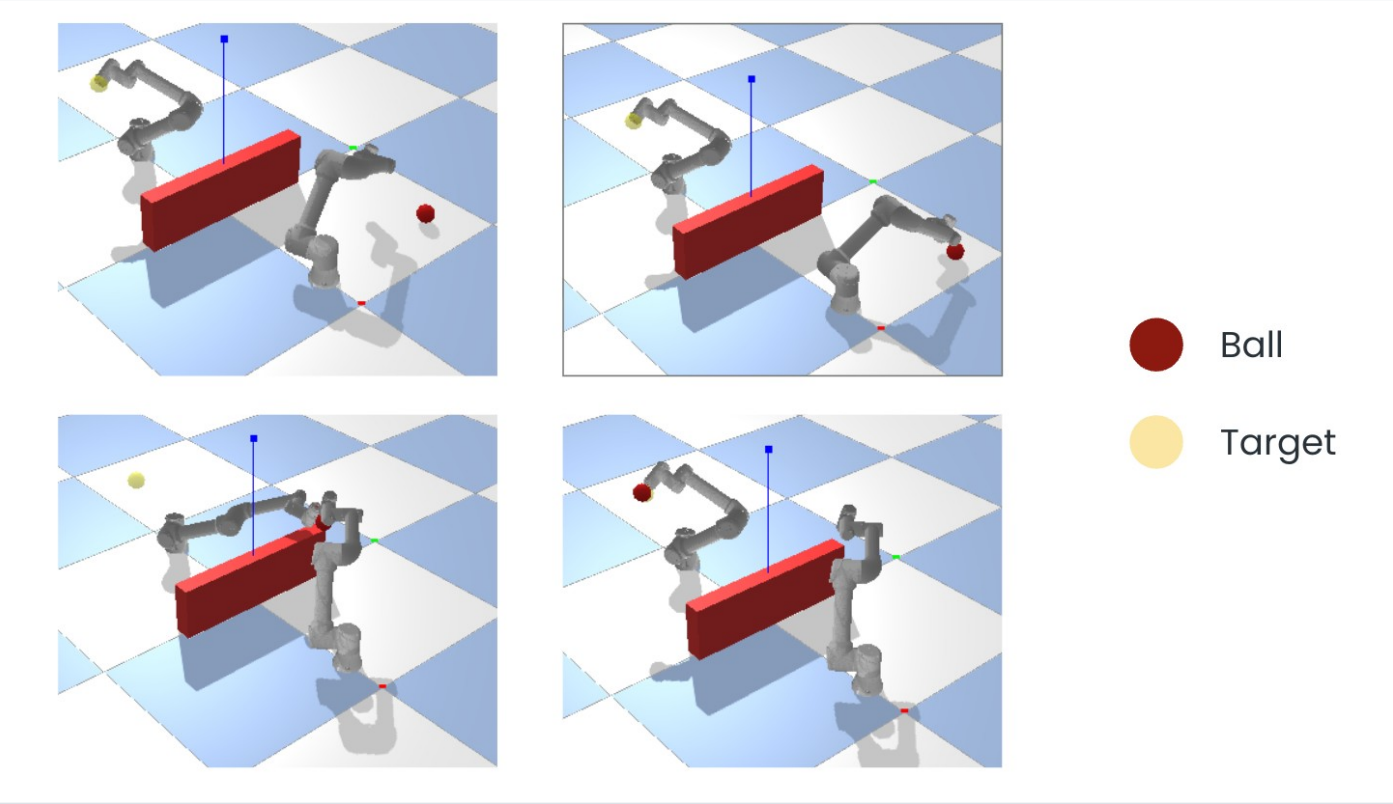


fig. 04 pick → handoff → place sequence; red wall forces the handoff

Code: github.com/bmc209/16350-Final-Project

From reset vector to servo control: a bare-metal Cortex-M4 kernel

I'm building a bare-metal kernel for an STM32 Cortex-M4 from the ground up. **I write every layer between the reset vector and user programs** — boot code, register-level drivers, interrupts, and syscalls — and use it to drive an autonomous robot car.

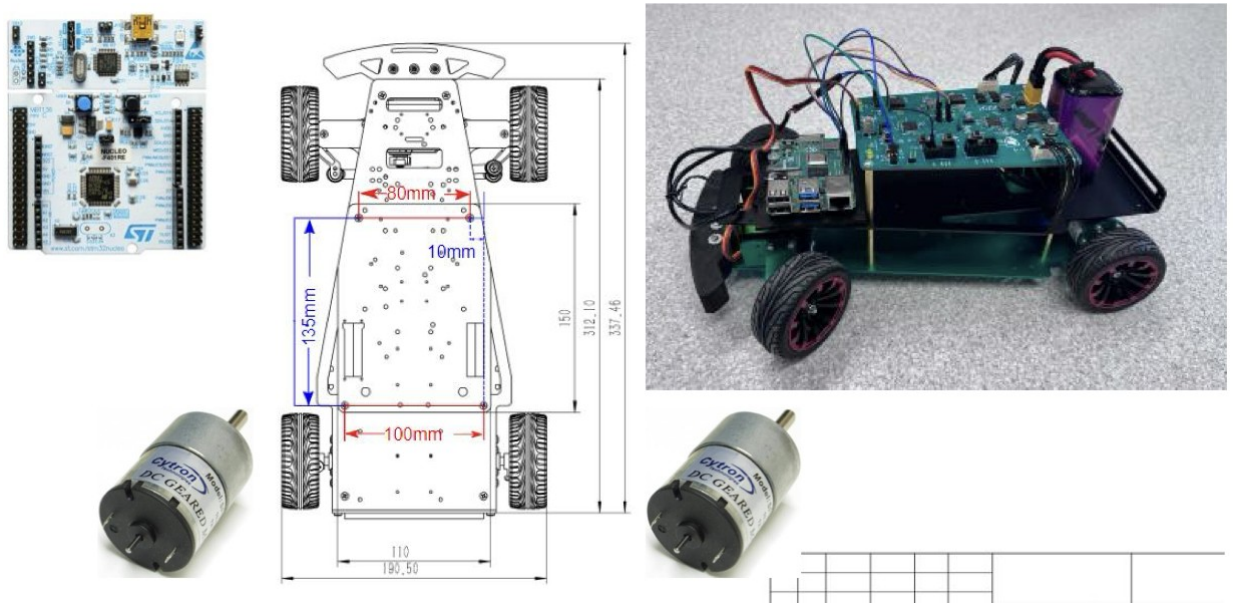


fig. 05 STM32 board, chassis, motors, and the assembled car

THE STACK I'M BUILDING (TOP = USER SPACE)

User programs · newlib

SVC syscall handler · user/kernel privilege separation

Interrupt layer: UART ring buffers · timer-IRQ servo PWM

MMIO drivers: UART · I2C · ADC · LCD

Assembly bootloader

STM32 · ARM Cortex-M4

Blue = layers I write

C | ARM Assembly | STM32 | MMIO | Interrupts | newlib

Game logic in silicon: a deterministic FPGA RTL system

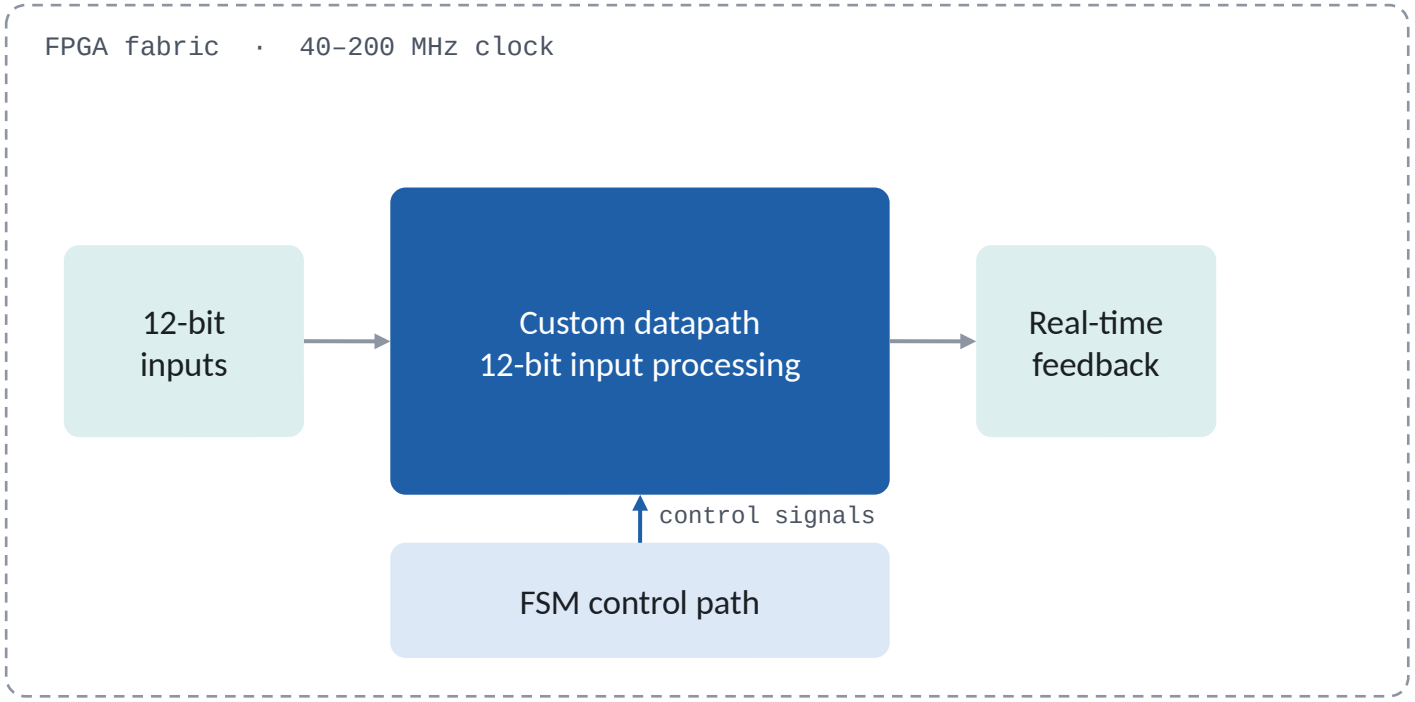


fig. 06 system block diagram (SystemVerilog, Vivado)

Software game loops jitter; hardware doesn't. **I designed an RTL system that runs game logic and real-time feedback directly in FPGA fabric**, with timing fixed to the clock cycle.

WHAT I BUILT

- FSM-based control path sequencing the game
- Custom datapath for 12-bit input processing
- Built in SystemVerilog and Vivado, running on FPGA

`SystemVerilog` `Vivado` `FSM design` `RTL`

12-bit
input datapath

40–200
MHz operating range

ns
cycle-level determinism

Karla Sahin · karlasahin0@gmail.com · klsahin.github.io

Xspine: 3D-printed structures that sense their own motion

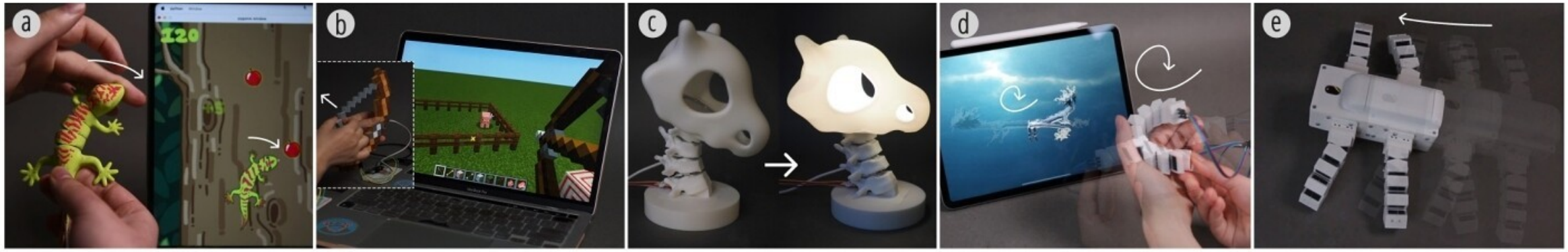


fig. 07 multi-material FDM structures as game controllers and interactive toys

Xspine embeds motion sensing directly into flexible multi-material 3D prints — no assembly. **I modeled their tactile and sensing feedback in software and built the pipeline that turns joint signals into training data** for AI models.

MY CONTRIBUTION

- Developed 5 application interfaces (fig. 07 a–e) that turn 8 joint signals streamed at 100 Hz into live game and toy input
- Virtually modeled force-tunable game controllers and fidget toys (Python, Unity, React)

CHI '26 — Full paper + interactive demo

doi.org/10.1145/3772318.3791317

UIST '25 — Adjunct poster

doi.org/10.1145/3746058.3758350

8 × 100 Hz — joint signals streamed

Leading labs, build teams, and the next generation of engineers

Teaching Assistant, Lab Lead

18-220 Electronic Devices & Analog Circuits · Jan 2026 – present

130+ students per semester; 2 weekly recitations + 3-hour labs, office hours, and hands-on circuit debugging

Senior Build Lead, ChemE Cube

CMU team · Jan 2025 – present

5th / 54 teams at the 2025 AIChE Annual Conference; I lead weekly builds of our Direct Air Capture system

SWE++ Curriculum Lead

Society of Women Engineers · Jan 2025 – present

20+ middle schoolers taught in an 8-week Python camp; wrote lessons and starter code, managed volunteers

Software & Perception, Lunabotics

NASA Lunabotics Competition, CMU team · Sep 2026 – present

NASA university robotics competition; I work on the team's software and perception

LET'S TALK

Seeking Summer 2027 internships in robotics & embedded software

karlasahin0@gmail.com
klsahin.github.io
github.com/klsahin
linkedin.com/in/karla-sahin

B.S. Electrical & Computer Engineering + Robotics (intended), Carnegie Mellon · May 2028 · GPA 3.85, Dean's List

LANGUAGES

C++ Python C SystemVerilog ARM Asm MATLAB Bash

ROBOTICS & SOFTWARE

ROS 2 Gazebo OpenCV Docker Linux CI/CD GDB pytest Grafana A* RRT/PRM MAPF

HARDWARE

STM32 FPGA/Vivado NVIDIA Orin PCB design UART/I2C/ADC SOLIDWORKS Oscilloscope